Práctica p6 Iker Núñez UO306425

El código resultante para el problema de Bin Packing es el siguiente:

Por una parte, la clase que ejecuta el algoritmo:

```java
// Solves the problem by backtracking
private void backtracking(int indexS, ArrayList<ArrayList<Integer>> distribution) {
    numberOfRecCalls++;

    // Base case
    if (indexS == setsS.length) {
        Integer thisK = distribution.size();
        if (thisK < bestK) {
            copySolution(distribution);
            return;
        }
    }

    // Pruning: if current size es greater or equal than best one, recursive call
    // ends
    // if (distribution.size() >= bestK) {
    //     // stops the execution
    //     return;
    // }

    for (int pos = 0; pos < distribution.size(); pos++) {
        // tries to add it to an existant container
        if (!setsFlag[indexS] && sum(distribution.get(pos)) + setsS[indexS] <= capacityC) {
            distribution.get(pos).add(setsS[indexS]);
            setsFlag[indexS] = true;
            backtracking(indexS + 1, distribution);
            setsFlag[indexS] = false;
            if (distribution.size() != 0) {
                distribution.get(pos).removeLast();
            }
```

```java
    for (int pos = 0; pos < distribution.size(); pos++) {
        // tries to add it to an existant container
        if (!setsFlag[indexS] && sum(distribution.get(pos)) + setsS[indexS] <= capacityC) {
            distribution.get(pos).add(setsS[indexS]);
            setsFlag[indexS] = true;
            backtracking(indexS + 1, distribution);
            setsFlag[indexS] = false;
            if (distribution.size() != 0) {
                distribution.get(pos).removeLast();
            }
        }
    }

    // could not add it to an existant one, creates a new container
    ArrayList<Integer> newContainer = new ArrayList<Integer>();
    newContainer.add(setsS[indexS]);
    setsFlag[indexS] = true;
    int indexNewInsertion = distribution.size();
    distribution.add(newContainer);
    backtracking(indexS + 1, distribution);
    setsFlag[indexS] = false;
    if (distribution.size() != 0) {
        distribution.get(indexNewInsertion).removeLast();
    }
}
```

Y por otra parte, la clase que mide los tiempos:

```java
Run | Debug
public static void main(String[] args) {
    String nombreFichero;
    long t1;
    long t2;
    double tResultado;

    System.out.println(x: "BACKTRACKING: BIN PACKING");

    System.out.println(x: "MEDICION DE TIEMPOS: ");
    for (int i = 0; i < 10; i++) {
        nombreFichero = "./test0" + i + ".txt";
        String[] argsMain = { nombreFichero };

        t1 = System.nanoTime();
        for(int j = 10; j < 1000; j++){
            AlmacenajeContenedores.main(argsMain);
        }
        t2 = System.nanoTime();
        tResultado = (t2 - t1) / 1_000_000.0;

        System.out.println("FICHERO: " + nombreFichero + ", TIEMPO(ms) = " + tResultado);
    }
    System.out.println(x: "MEDICION DE TIEMPOS FINALIZADA");
}
```

Tabla de medición de tiempos:

fichero	S	t(ms)
test00	10	33,6597
test01	12	3,2356
test02	10	1481,4852
test03	12	34,2125
test04	15	fdt
test05	20	fdt
test06	14	fdt
test07	15	fdt
test08	20	fdt
test09	20	fdt

A pesar de que los tiempos salen un poco extraños, para el test04.txt se tardo +5 min en ejecutar. Esto es porque en los algoritmos de backtracking, la complejidad es exponencial.

En futuras prácticas, veremos algunas técnicas como poda u ordenar el array de entrada y con estas podremos disminuir la complejidad en gran parte.